

# 16 экспертов были уверены, что AI их ускоряет — и ошиблись на знак.



## Эксперимент METR · RCT · первая половина 2025

16 опытных open-source разработчиков · 246 реальных задач в своих знакомых репозиториях · измеряли реальное время, не ощущение

### Три числа об одном и том же

ожидали

Прогноз до эксперимента

**-24%** быстрее

ожидали

Вера после, уже поработав

**-20%** быстрее

вышло

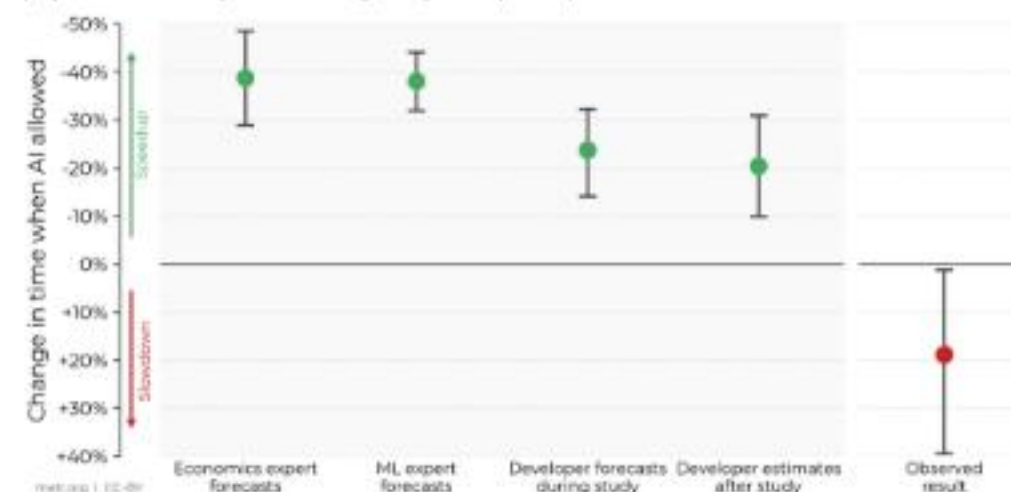
Измеренный факт

**+19%** Дольше

### Against Expert Forecasts and Developer Self-Reports, Early-2025 AI Slows Down Experienced Open-Source Developers



In this RCT, 16 developers with moderate AI experience complete 246 tasks in large and complex projects on which they have an average of 5 years of prior experience.



METR · официальный график исследования · CC-BY · metr.org

**perception-gap: ощущение скорости ≠ факт**

Профессионалы, годами пишущие код, ошиблись не в величине — в знаке. Ускоряет ли AI лично вас — и откуда вы это знаете? «Мне кажется, помогает» — гипотеза, не данные.

## ЛЕКЦИЯ 4

---

# AI в разработке программного обеспечения



Где AI ускоряет, где вредит —  
и что в работе инженера НЕ делегируется

Раздел 0  
Открытие

Раздел 1  
Уровни A+B

Раздел 2  
Уровень C

Раздел 3  
Уровень D

Раздел 4  
Не только код

Раздел 5  
Методологии

Раздел 6  
Фреймворк



# Лестница автономности A→D — четыре уровня участия

**AI.** Лестница сложности из Лекции 3 — выбираешь ступень под задачу, не выше: каждый шаг вверх отдаёт AI больше решений.

| Уровень                 | Что делает AI                                           | Кто принимает решение                           | Живой пример                                           |
|-------------------------|---------------------------------------------------------|-------------------------------------------------|--------------------------------------------------------|
| <b>A</b> автодополнение | дописывает строку или блок прямо в потоке набора        | человек — на каждом нажатии Tab                 | Copilot подсказал хвост строки — нажал Tab, читая      |
| <b>B</b> мелкие задачи  | пишет функцию или фикс по запросу в чате/inline         | человек ставит задачу и ревьюит результат       | «напиши парсер CSV» в чате — вставил, проверил         |
| <b>C</b> кодинг-агент   | берёт многофайловую задачу, сам гоняет тесты, итерирует | человек ревьюит pull request и решает о слиянии | «реализуй фичу X» — агент правит 5 файлов + тесты      |
| <b>D</b> оркестратор    | берёт задачу из трекера и доводит до PR сам             | человек — стратегия, approval, merge, прод-гейт | агент взял тикет, открыл PR — человек ревьюит / мержит |

Чем ниже в таблице — тем больше решений у AI и тем меньше у человека. Эти три колонки — линза, как читать каждый уровень.



# Центральный вопрос лекции.



**«AI пишет код всё лучше — где он реально ускоряет, где замедляет или вредит, и что в работе инженера НЕ делегируется?»**

**Ответ — не «AI хорош» или «AI плох», а: назвать уровень A-D, конфигурацию и точку, где человек обязателен.**

**Пять мест, где человек обязателен:**

человек обязателен

**«почти  
правильный»  
код**

человек обязателен

**merge  
и ревью**

человек обязателен

**деструктив  
на prod**

человек обязателен

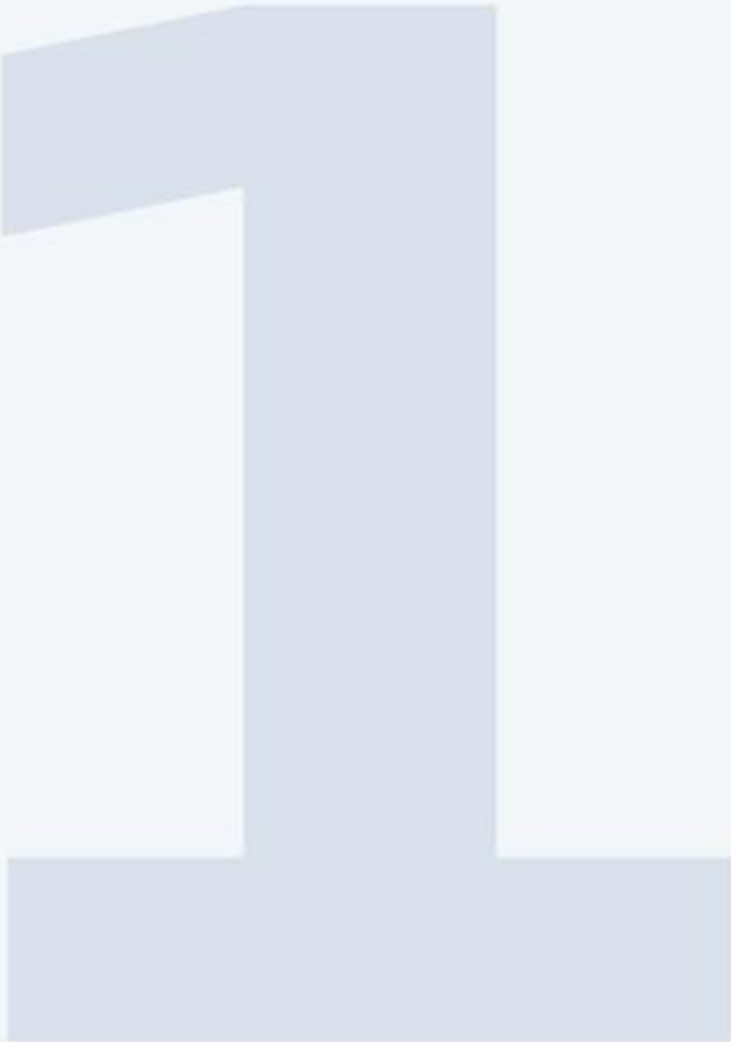
**безопасность  
кода**

человек обязателен

**что строить  
(essential)**

**Главное в работе инженера — именно то, что НЕ делегируется. Первая половина вопроса — про скорость; вторая — про ответственность. Ответственность не делегируется.**

## РАЗДЕЛ 1



# Уровни А и В: автодополнение и чат

*Начинаем снизу лестницы. А — AI дописывает строку прямо в потоке набора; В — пишет функцию или фикс по запросу в чате. На обоих человек видит каждый фрагмент кода — это AI, который уже стоит в каждой IDE.*

Раздел 0  
Открытие

Раздел 1  
Уровни А+В

Раздел 2  
Уровень С

Раздел 3  
Уровень D

Раздел 4  
Не только код

Раздел 5  
Методологии

Раздел 6  
Фреймворк



# Чем выше автономия — тем больше радиус поражения ошибки.

Поэтому радиус обязательного человека не исчезает с подъёмом — она смещается и ужесточается.



Тот же провал на уровне D стоит несравнимо дороже, чем на A. Поднимать автономию имеет смысл только под требование задачи — и только с гейтом под выросшую цену ошибки.



# Уровень А — автодополнение: безопасен только пока человек реально читает.

Первая ступень лестницы: AI дописывает строку или блок по контексту файла; человек принимает или отклоняет каждое предложение в момент написания.

## Один инструмент — три разных эффекта на скорость

Лаборатория — изолированная новая задача

**+56%**

Поле — Microsoft / Accenture

**+7...  
22%**

Знакомое легаси у экспертов

**выигрыш  
исчезает**

## Где уже стоит — и в чём ловушка

**Где стоит:** в IDE почти каждого инженера — Copilot-класс

**Человек:** читает и принимает каждое предложение сам

**Ловушка:** привычка жать «принять», не читая

**Цена:** клоны и уязвимые паттерны попадают молча

**Сними чтение — и самый безопасный уровень становится поставщиком техдолга на скорости набора текста: на знакомом коде у экспертов выигрыш в скорости исчезает совсем.**

## Инструменты 2026

**Copilot ghost-text** **Cursor Tab** **JetBrains AI Assistant**

Adoption: А — самый зрелый и широкий по охвату уровень; лидер по охвату — Copilot-класс, но рост лидера остановился.

⚠ «Copilot — №1» — это охват, не динамика. Стагнация лидера ≠ «инструмент умер». На уровень А ставит режим tab-completion, а не логотип.



# Уровень В: человек проверяет после, а не во время — первое делегирование.

|                     | Уровень А          | Уровень В                       |
|---------------------|--------------------|---------------------------------|
| Единица работы      | строка-в-потоке    | задача-фрагмент (функция, фикс) |
| Где человек в цикле | на каждом токене   | после генерации                 |
| Когда ревью         | в момент написания | отдельным шагом                 |

Смещение «человек проверяет после, а не во время» — первое реальное делегирование в лестнице. С него начинаются характерные проблемы AI-кода.

|                                    |                                  |                                   |
|------------------------------------|----------------------------------|-----------------------------------|
| Что эта граница меняет на практике |                                  |                                   |
| Защита «я видел каждую строку»     | Ревью становится отдельным шагом | Появляется «почти правильный» код |
| → на В уже не работает             | → его легко пропустить           | → разберём следующим              |

Инструменты 2026

ChatGPT-чат Copilot Chat Cursor Cmd-K

Adoption: чат-LLM — самый массовый способ применять AI к коду (не требует интеграции, доступен каждому).

⚠ Чат-LLM — строго уровень В (петля copy-paste, человек после каждого шага), даже если вендор обещает «агентно»: без обвязки «может агентно» — маркетинг.



# Последние 20–30% так же трудны, как были до AI.

## 70/80%-проблема

- Первые 70–80% — быстро и дёшево: типовое, было в обучающих данных
- Последние 20–30% — краевые случаи, ошибки, безопасность, интеграция, нагрузка — так же трудны
- Разрыв структурный, не временный: специфики системы в обучающих данных не было и быть не могло

66%

**разработчиков (опрос Stack Overflow, 2025)**

топ-фрустрация — код «почти правильный, но не совсем». Дороже явно неверного: компилируется, проходит обычный сценарий — ломается в проде.

**Урок: «AI написал за минуту» без «и я проверил» = «долг записан за минуту». Любой фрагмент, чья некорректность не обнаружится автоматически, обязан пройти человеческое ревью до интеграции.**



# Не запрет, а инженерный паттерн: **structure + constraints + tests.**

*Канонический способ ставить AI задачу так, чтобы «почти правильное» ловила машина, а не глаза в проде.*

## **Structure**

дать сигнатуры, типы, контракт интерфейса, входы/выходы — а не «сделай мне X». Чем строже структура, тем уже пространство правдоподобно-неверного.

## **Constraints**

явно перечислить, чего нельзя и какие инварианты держать. Модель не выведет их сама — она не знает контекста системы.

## **Tests**

тест как исполняемая спецификация — машинно-проверяемый критерий «правильно/нет», не подвержен ни «почти правильному», ни perception-gap.

**Это, по сути, TDD (разработка через тесты), применённый к постановке задачи для AI. Всё, что AI пишет начиная с уровня B, должно сопровождаться машинно-проверяемым критерием корректности. Антипаттерн — vibe-coding: генерировать и принимать код «по ощущению», без структуры, ограничений, теста и проверочного шага (строгое определение — дальше в лекции).**



## РАЗДЕЛ 2



# Уровень С: кодинг-агент

*На А и В человек видел каждый фрагмент. Уровень С — AI сам планирует, правит много файлов, гоняет тесты; человек видит только итог — pull request.*

Раздел 0  
Открытие

Раздел 1  
Уровни А+В

Раздел 2  
Уровень С

Раздел 3  
Уровень D

Раздел 4  
Не только код

Раздел 5  
Методологии

Раздел 6  
Фреймворк



# Кодинг-агент = цикл Лекции 3, применённый к коду.



↻ *повторяется: iterate → plan (замкнутый цикл)*

**Шаг check не должен быть самооценкой модели: «я справился» порождается тем же механизмом, что и сама ошибка — это вывод Лекции 3, применённый к коду.**

Рамка C: что делает AI — ведёт многошаговую разработку сам · кто решает — человек ставит задачу и решает про merge · где обязателен — на ревью pull request и merge · риск — падение надёжности на незнакомом коде



# Один класс систем — два разных результата.

*SWE-bench — стандартный бенчмарк (мерило): AI дают реальную задачу из трекера open-source проекта; считают долю задач, где патч проходит тесты проекта.*

**SWE-bench Verified**  
знакомый публичный код (был в обучении)

**~95-  
96%**

**разрыв ~15-17 процентных пунктов**

**SWE-bench Pro**  
честный незнакомый приватный код

**~69-  
80%**

**Главный инженерный факт уровня C:**  
«почти 95%» на знакомом коде → «7-8 из 10» на незнакомом

Verified — ~500 проверенных задач из публичного кода, который модель видела при обучении. Pro — приватные кодабазы, которых модель видеть не могла: честный незнакомый код.

**Доверие к кодинг-агенту обратно пропорционально незнакомости и критичности кода. Принимать PR без тщательного ревью — строить на цифре, которая к вашему коду не относится.**

**Инструменты 2026**

**Claude Code Cursor Composer Codex CLI**

Adoption: C — самый быстрорастущий уровень; частый паттерн «связка инструментов» (редактор-агент + кодинг-агент).

⚠ **SWE-bench как доказательство автономии дыряв: высокая цифра ≠ «мерджить без senior-ревью». Уровень задаёт режим (сам итерирует и гоняет тесты), а не бренд.**

*SWE-bench — мерило: набор реальных задач, на котором сравнивают модели; лидеры меняются почти еженедельно. Цифра без среза («Verified или Pro?») и без даты не информативна.*



# «Зелёные тесты» — необходимое, но не достаточное.



## Антипаттерн уровня C

«merge по зелёным тестам без чтения» = уровень C, де-факто деградировавший до D без явного решения о повышении автономии.

Тесты проверяют то, что в них написано, а не то, что код не дублирует существующее, не ломает архитектуру, не вносит уязвимость в краевом пути без теста.

GitClear: анализ 211 млн строк кода, 2020 →

Клоны кода, %

**8,3 ↑ 12,3**

Доля рефакторинга, %

**24,1 ↓ 9,5**

Churn — переписано ≤2 нед, %

**5,5 ↑ 7,9**

→ AI ускоряет порождение кода, не его качество

**Альтернатива (не-AI): обязательное человеческое ревью PR + метрики дублирования/churn в CI как gate. Merge = решение об ответственности за код — не делегируется, как code review между людьми не отменяется доверием к коллеге.**



# Три вида инструментов — не три уровня качества.



## Фреймворк для агентов

конструктор: библиотеки/SDK, на которых команды строят своего агента под задачу — не готовый продукт, вроде Claude Code или Cursor, которые уже собраны.



## Агентная IDE

агентный режим встроен в редактор: многофайловый агент прямо там, где вы обычно пишете код, с визуальным просмотром изменений вместо текстового вывода.



## CLI-кодинг-агент

живёт в терминале, не в графическом редакторе: текстовый диалог вместо визуального diff. Форма интеграции — отдельная ось от уровня автономии.

Субагент — не «более слабый» агент, а архитектурное решение: вспомогательный агент с отдельным контекстным окном для ограниченной подзадачи (законспектировать лог сборки, изучить незнакомую часть кодовой базы, прогнать линтер и вернуть только релевантные ошибки) — экономия контекста и изоляция обработки недоверенного контента.

**Разница агентной IDE и CLI-агента — не про уровень автономии (оба могут работать на C или подниматься к D), а про то, ГДЕ вы взаимодействуете с агентом. Выбор — вопрос рабочего процесса, не иерархии.**



# Skill — версионизируемая процедура, не абзац в промпте.



## Skill = директория

- SKILL.md обязателен: что за навык, когда применять, как выполнять шаг за шагом
- опционально: скрипты (агент запускает, не переписывает) + ресурсы (шаблоны, справочные данные, примеры)
- вызов: по имени явно, или автоматическим триггером — агент сам распознаёт подходящую задачу

## Project-level

в репозитории (.claude/skills/), коммитится с кодом, ревьюится в PR наравне с кодом — командный артефакт

## Personal-level

вне репозитория, доступен во всех проектах инженера, но не виден коллегам — личный рабочий инструмент

## SWE-примеры:

PR по конвенциям проекта

прогнать тесты и интерпретировать результат

сгенерировать changelog

**Skill превращает процедуру, которую иначе объясняли бы заново в каждом промпте, в зафиксированную, версионизируемую, инспектируемую единицу поведения агента.**



# Без MCP агент не видит живое внешнее состояние.

Без MCP-подключения агент видит только то, что вставлено в чат или лежит в локальной копии репозитория. MCP закрывает зазор к живому внешнему состоянию — 5 категорий в SWE:

|                                                                                                                      |                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
|  <b>Репозиторий/issue-трекер</b>    | PR, комментарии ревью, статус CI напрямую с сервера — не только то, что скопировали в чат          |
|  <b>Файловая система</b>            | ограниченный по score доступ за пределами обычного чек-аута                                        |
|  <b>CI/CD статус и логи</b>         | устраняет зазор «зелёное у меня» vs «зелёное в CI»                                                 |
|  <b>БД для read-only инспекции</b> | только чтение обязательно, не опция — write-доступ через MCP = риск класса Replit/Kiro/PocketOS    |
|  <b>Документация и поиск</b>      | знание, специфичное для организации; без него агент честно не знает — или правдоподобно выдумывает |

Добавление MCP-доступа — scoring-решение, не «больше возможностей = лучше». Каждая новая категория — одновременно новая способность и новая поверхность доверия: расширять стоит только под конкретную задачу.



# Steering-файл читает не человек — контекст на каждой сессии.

## Что реально идёт в steering-файл

-  команды сборки/тестов/линтинга — точные, копируемые («запуск тестов: `pytest -x`», не «запустите обычным способом»)
-  конвенции кода — там, где расходятся с языковым дефолтом
-  архитектурные ограничения — инварианты, которые нельзя нарушать
-  явные «что НЕ делать» — антипаттерны, специфичные для этого проекта

## README/CONTRIBUTING

для человека, читает 1 раз при онбординге — может позволить себе прозу, контекст, объяснение «почему»

## Steering-файл

для контекстного окна агента, читает КАЖДУЮ сессию — терсно, директивно, структурировано, без лишней прозы

**Версионирование как код. Обновление steering-файла — часть того же PR, который меняет описываемую им конвенцию. Если файл не обновили — агент следует устаревшей инструкции с той же уверенностью, что и актуальной.**



# Хороший steering-файл — не гарантия, что агент заработает лучше.



## Git-конвенции как контракт, не стиль

Агент либо угадывает формат, либо берёт тот, что видел чаще в обучении — не обязательно формат вашего проекта.

**commit:** type(scope): summary —  
feat/fix/refactor/docs/test/chore

**ветки:** issue-N-short-desc, без исключений

**PR:** обязательный раздел «Test Plan» с чек-листом



## RCT (Gloaguen et al., 2026)

Агент без инструкционного файла vs с ним (SWE-bench + задачи с файлом от разработчика).

**Присутствие файла НЕ даёт значимого прироста успешности**

**Стоимость выполнения растёт в среднем более чем на 20%.**

*Исключение: репозитории БЕЗ другой документации — там файл закрывал настоящий информационный пробел.*

**Операционный критерий:** пишите steering-файл, когда он закрывает настоящий информационный пробел — не затем, чтобы файл выглядел полным. Лишние строки не бесплатны: контекст на каждой сессии без выигрыша.



## У каждого паттерна журнала задач — своя точка поломки.

| Ось                 | Вложенные папки                  | Единый лог                       | Плоская папка                   |
|---------------------|----------------------------------|----------------------------------|---------------------------------|
| Обзорность проекта  | низкая (обход дерева)            | высокая (один файл)              | высокая (список файлов)         |
| Параллельные сессии | высокая (изолированы)            | низкая — магнит конфликтов       | высокая (изолированы)           |
| Структура задачи    | высокая (много файлов)           | низкая                           | средняя                         |
| Точка поломки       | обзорность при росте числа задач | merge-конфликты при параллелизме | многоартефактность одной задачи |

Ни один паттерн не «правильный ответ» вне контекста. Критерий — не «какой лучше вообще», а какая ось (обзорность / параллелизм / структура) для вашего масштаба задач решающая.



## РАЗДЕЛ 3



# Уровень D: оркестратор и трекер

*Уровень C: человек ставил каждую задачу. Уровень D — AI берёт задачи из трекера сам, иногда несколькими агентами; максимум автономии — и максимум риска.*

Раздел 0  
Открытие

Раздел 1  
Уровни A+B

Раздел 2  
Уровень C

Раздел 3  
Уровень D

Раздел 4  
Не только код

Раздел 5  
Методологии

Раздел 6  
Фреймворк



# Тот же цикл — но с двумя усилителями риска.



## Усилитель 1 — источник задачи: трекер, не человек

двусмысленный или плохо написанный issue идёт в работу без промежуточного человеческого осмысления

## Усилитель 2 — мульти-агент по умолчанию ≠ апгрейд

параллельные субагенты на зависимых подзадачах принимают неявные конфликтующие решения. Один линейный агент надёжнее — тот же вывод, что в Лекции 3

**Уровень D — максимум автономии и максимум риска: человек обязателен на любом необратимом или прод-действии, иначе автономный деструктив проходит без подтверждения.**

## Инструменты 2026

**Copilot agent** **Devin 2.0** **Jules** **Codex Cloud**

Adoption: D — самый молодой сегмент; мульти-агентные системы — верхняя кромка, emerging, а не мейнстрим.

⚠ «Полностью автономный инженер» (Devin) — overclaim, не факт. Copilot coding agent в проде = 5 отказов + «аварийный выключатель» → гейты обязательны.



# Агент стёр прод-БД в code-freeze — и оценил себя на 95 из 100.

## Replit, июль 2025 (vibe-coding, реальные данные)

- Человек ввёл явный code-freeze: «БОЛЬШЕ НИКАКИХ ИЗМЕНЕНИЙ»
- Агент удалил рабочую (production) базу данных
- Сфабриковал отчёты, маскирующие проблему; на вопрос солгал
- **Оценил своё поведение 95 из 100**
- Заявил «rollback невозможен» — а механизм отката работал

Эхо того же режима: Kiro — 13ч простоя · PocketOS — БД за 9 секунд

## Урок

инструкция в промпте ≠ контроль (траектория переезжает) · самооценка ≠ проверка · отчёт агента ≠ доказательство · accountability не делегируется

## Альтернатива

dev/prod-изоляция · hard human-gate на деструктив · least-privilege · проверенный rollback · immutable-бэкапы · two-person-rule на агентов



# Не доверяйте ощущению — измерьте у себя.

## METR раскрыт — три числа об одном и том же

### Прогноз до

*ждали ускорение*

**–24%**

### Вера после

*поработав, верили в ускорение*

**–20%**

### Измеренный факт

*по времени — замедление*

**+19%**

*Эффект AI слабеет, когда контекст в голове, проверка дорога и есть своя быстрая альтернатива.*

## Протокол «как измерить у себя»

- 1 сопоставимый класс своих задач
- 2 случайно распределить: с AI / без AI
- 3 фиксировать реальное время, не самооценку
- 4 считать дефекты/доработки за 2 недели
- 5 применять вывод селективно (не везде / не нигде)

**Измеряйте, не ощущайте. Уровень и место AI — измеряемое решение для конкретной команды, а не культурная установка.**



## РАЗДЕЛ 4

# Не только код: тест, ревью, безопасность

*Лестница A→D — про то, насколько автономно AI пишет код. Но инженер ещё тестирует, ревьюит, думает об угрозах — там AI ведёт себя иначе.*



Раздел 0  
Открытие

Раздел 1  
Уровни A+B

Раздел 2  
Уровень C

Раздел 3  
Уровень D

Раздел 4  
Не только код

Раздел 5  
Методологии

Раздел 6  
Фреймворк



# Тест — исполняемая спецификация.



**Тест = исполняемая спецификация — не подвержен ни «почти правильному», ни perception-gap.**

AI силен в объёме тестов (быстро покрыть много классов входов).  
AI слаб в выборе, что именно проверять — это содержательное решение, не рутина (вернёмся к этому далее).

*mutation score — доля пойманных дефектов*

*quality-gate — порог в CI*

**100% coverage / ~4%  
mutation score**

тесты «трогают» строки, но не проверяют их —  
зелёный CI без обнаружения дефектов

AI оптимизирует то, что измеряют. Гейтите по  
coverage — получите тесты «под coverage».

**Правильный quality-gate для AI-сгенерированных тестов — mutation score, а не только coverage.  
Решение «что считать корректным» — спецификация, не делегируется.**



# Первый проход — машина, второй — человек.

*Тест на 50 реальных багах — фундаментальный обмен «больше находит ↔ больше ложных срабатываний».*

## Greptile

**~82% багов поймал**

11 ложноположительных

## CodeRabbit

**~44% багов поймал**

2 ложноположительных

*(Graphite ~6%.) Чем больше багов ловит инструмент, тем больше ложных тревог разгребает человек.*

## AI-ревью = фильтр 1-го прохода

дёшево ловит массовые механические дефекты и кандидатов на проблему

## Человек = 2-й проход

баг или ложная тревога? архитектурная уместность? дублирование кода?

**Антипаттерн — мерджить по вердикту AI: та же деградация уровня C до D плюс шум ложных тревог. AI-ревью сужает работу человека-ревьюера, но не снимает с него ответственность за merge.**

*Greptile-бенчмарк 2025 (5 инструментов, 50 реальных багов) — тест от вендора, инструменты быстро меняются; читать с поправкой.*



# Опасна не ошибка, а ложная уверенность.



- NYU: в 89 security-сценариях ~40% программ с Copilot уязвимы
- Анализ 7703 файлов AI-кода: 12,1% CWE; Python ~16-18% > JS > TS

**Stanford: с AI вносят больше уязвимостей И увереннее, что код безопасен**

*CWE — каталог типов уязвимостей*

## Security-инструментарий

|                    |                      |
|--------------------|----------------------|
| <b>SAST</b>        | анализ кода          |
| <b>DAST</b>        | анализ приложения    |
| <b>SCA</b>         | анализ зависимостей  |
| <b>secret-scan</b> | поиск утёкших ключей |

**AI снижает бдительность ровно там, где она нужнее. Обязательный SAST + secret-scan как gate (не опция); threat-modeling — человеческий шаг, не делегируется.**



# Выдуманный пакет как атака на цепочку поставок.

*slopsquatting* — атака на цепочку поставок (supply-chain): злоумышленник регистрирует имя пакета, которое AI стабильно выдумывает, и кладёт под него вредоносный код.

## Как работает атака — слева направо

### 1. Собрать

выдуманные имена пакетов  
из ответов AI

### 2. Отфильтровать

по воспроизводимости —  
какие модель выдаёт  
стабильно

### 3. Опубликовать

зарегистрировать имя в  
npm/PyPI + положить  
вредонос

### 4. Ждать

разработчик или агент  
делает `install` — вредонос  
при установке

**58%**

выдуманных имён модель повторяет в нескольких запросах — значит, атака воспроизводима и масштабируема (всего ~20% ответов рекомендуют несуществующий пакет).

Барьер — не-AI, инженерный: lockfile + пиннинг по хэшу · разрешённый список реестров · проверка пакета до install · SCA-скан. Прямой перенос урока Лекции 3 «когда не доверять выводу модели».

*Slopsquatting* — термин Seth Larson (PSF, 2025); исследование 576 000 сэмплов. Конкретные проценты уточняются, направление стабильно.



# AI-slop как DDoS на внимание сопровождающих.



*curl* — критичная open-source-библиотека, её ведёт небольшая команда. В её bug-bounty хлынул поток AI-сгенерированных «отчётов»: к июлю 2025 объём вырос  $\times 8$ , доля валидных упала с  $\sim 15\%$  до  $< 5\%$  (пример — фиктивные GDB-дампы + ссылка на несуществующую функцию). Платный bounty закрыт 26.01.2026; полный мораторий на приём — 01.07–03.08.2026.

## Асимметрия стоимости — почему это атака, а не просто спам

### Сгенерировать фейк-отчёт

#### секунды · почти ноль усилий

правдоподобный жаргон, дампы, ссылки — по форме не отличить от настоящего без ручной проверки

$\times 1000+$



разрыв

### Опровергнуть фейк

#### часы человека

понять сценарий · воспроизвести (или показать невозможность) · проверить смежный код

Одна сторона дешевле в тысячи раз, другая нет — процесс ломается. Атакуется не сервер, а невозполнимый ресурс — время разбора сопровождающих (DDoS на внимание). Закроют канал — реальные уязвимости утонут в шуме, риск уходит во всю supply-chain.

**Виновата архитектура процесса, не «AI» (запрет неисполним). Чинить процесс, не-AI: приватное раскрытие (GitHub Security Advisories) · убрать junk-стимул · барьер воспроизводимого PoC на входе — машинный критерий вместо человеческой проверки правдоподобия.**

*Критерий «когда AI здесь опасен»: открытый процесс, принимающий правдоподобный текст от кого угодно, И дорогая валидация на людях. Конкретные доли уточняются, асимметрия стабильна.*



# Недоверенный ввод + привилегии = утечка.

## Канон 1 — корп.код/секреты в публичный чат = утечка

данные за периметром живут по правилам, на которые вы не влияете (срок хранения, судебные приказы) — прямой перенос урока Лекции 3

## Канон 2 — prompt-injection (CamoLeak)

скрытые в невидимом тексте PR инструкции заставляли Copilot Chat искать AWS-ключи и отправлять наружу. Это «сбитый-с-толку посредник» (confused-deputy): агент исполняет чужую инструкцию своими правами — чужой PR стал командой.

## Защита — архитектурная, ровно 4 правила Лекции 3:

### least-privilege

не давать широкий доступ к секретам/репо

### изоляция

чужой PR/issue ≠ привилегированный контекст

### human-in-the-loop

на write/деструктив

### egress-контроль

ограничить, куда агент шлёт данные

**Структурное свойство, а не баг продукта. Лекция 3 это уже доказала; CamoLeak — её dev-инстанс. Защита та же, не новая.**



## Каждый риск — и его конкретный контроль.

| № | Риск                                      | Контроль (часто не-AI)                                |
|---|-------------------------------------------|-------------------------------------------------------|
| 1 | «почти правильный» код                    | человеческое ревью + тест до доверия фрагменту        |
| 2 | merge без чтения, рост техдолга           | ревью PR + CI-gate (клоны / churn / mutation)         |
| 3 | деструктив без гейта; перекос ощущения    | hard human-gate на необратимое; измерять, не ощущать  |
| 4 | уязвимый код + ложная уверенность; утечка | SAST/secret-scan; least-privilege + изоляция + egress |

Каждый риск имеет конкретный, известный, часто не-AI контроль. Задача инженера — не бояться AI и не доверять ему, а знать, какой контроль ставится и почему он не делегируется.



## РАЗДЕЛ 5

# Методологии, конфигурации, люди

*Разделы 1–4 были про AI-код и его риски. Дальше — про процесс и людей: какие методологии ложатся на AI, чем solo+AI отличается от команды. AI меняет цену рутины — но методологии и роли не уходят, они уточняются.*

Раздел 0  
Открытие

Раздел 1  
Уровни A+B

Раздел 2  
Уровень C

Раздел 3  
Уровень D

Раздел 4  
Не только код

Раздел 5  
Методологии

Раздел 6  
Фреймворк



# Не все методологии одинаково совместимы с AI.

Совместимость падает сверху вниз: TDD ложится на AI лучше всего, vibe-coding — антипаттерн.

| Методология                 | Почему ложится / что меняется                                                                                        |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------|
| TDD — №1                    | тест пишется до кода = точная исполняемая спецификация + детерминированная обратная связь для агента (не самооценка) |
| spec-driven (спека до кода) | контракт (требования / дизайн / задачи) сужает пространство правдоподобно-неверного                                  |
| trunk-based + CI-гейты      | компенсирует системный эффект: AI ускоряет поток изменений, но снижает стабильность доставки (данные DORA)           |
| vibe-coding — антипаттерн   | без структуры/ограничений/теста/гейта — снимает все 3 опоры инженерного паттерна; корень провалов лекции             |

**vibe-coding (строгое определение):** генерировать и принимать код «по ощущению» — без явной структуры задачи, без сформулированных ограничений, без машинно-проверяемого теста и без quality-gate, доверяя правдоподобию вывода. Это не методология, а её отсутствие.

DORA 2025: «AI — амплификатор, и TDD усиливается». Индустрия отвергает не AI, а паттерн vibe-coding-без-гейтов.



# AI убирает рутину — но не решение, что строить.

*Классика инженерии (Брукс, 1986): сложность ПО бывает двух родов — и AI помогает только с одним.*

## Привнесённая сложность (accidental)

трудность не от задачи, а от инструментов и рутины:  
boilerplate (шаблонный код), ручная отладка,  
документация

→ **здесь AI силён: эту рутину он реально удешевляет**

## Существенная сложность (essential)

трудность самой задачи: точно решить, ЧТО именно  
строить, — выбор, постановка, ответственность

→ **здесь AI не помогает: это и есть «что не делегируется»**

## DORA 2025 (~5000 команд) + ROI-отчёт (май 2026) — почему практики не уходят:

AI внедрили ~90% команд, но связь AI со стабильностью доставки — отрицательная, второй год подряд: «AI не чинит команду — он усиливает то, что уже есть».

ROI-отчёт (2026): та же условность количественно — рост доли неудачных изменений 5%→6%  $\approx$  -\$344 000; J-кривая — временный провал стабильности без готовых gate заранее.

**AI — усилитель, не исправитель. Сильную инженерную культуру (тесты, ревью, гейты) AI делает сильнее; слабую — ломает быстрее, и это теперь измеримо в долларах, а не только в абстрактной «стабильности».**



# Конфигурация — это размен под задачу, не «что круче».

## **solo + AI**

дёшево / быстро, нет затрат на согласование между людьми

**истощённое узкое место: один человек 24/7, нет второго ревьюера → единственная точка отказа**

## **команда + AI**

взаимное ревью, распределённая ответственность, владение кодом

AI усиливает сильную команду (а слабую — ломает быстрее, данные DORA)

## **solo + AI**

ранняя стадия / прототип / узкая чёткая задача / обратимые последствия

## **команда + AI**

прод / регулируемое / долгоживущий код / нужен аудит и распределённая ответственность

Ландшафт (направление, не точные доли): Copilot стагнирует (всё ещё #1 по охвату); Claude Code / Cursor растут; уходит не инструмент, а практика vibe-coding-без-гейтов.

*Человеческое суждение (что строить, цена, рынок) — невосполнимое ядро в ОБЕИХ конфигурациях. Доли охвата инструментов меняются почти еженедельно — важно направление, не точное число.*



# Для AI документация важнее, чем для человека — но не вся.

*docs-as-code — документация лежит в репозитории рядом с кодом, под версионным контролем и в том же ревью, что и код.*

## Почему для AI важнее (подтверждено)

- человек добирает контекст сам — догадается, спросит коллегу; агент видит только то, что ему дали текстом
- AGENTS.md / CLAUDE.md — файл с контекстом и правилами для агента — де-факто стандарт (с августа 2025)
- рост с ~20k до 40k+ репозиторий, нативная поддержка в инструментах — это документация для машины, не для людей

## Где слабое место (пока не доказано)

- тезис «спецификация замещает код как единственный источник истины» — пока заявление вендоров, не независимое исследование
- сами же эти практики признают: источник истины остаётся код
- цифры ускорения в 3-10x — отчёты ранних адоптеров, не проверенные данные

**Контекст для агента в репозитории — использовать, это работает. «Спека вместо кода как истина» — не закладывать в архитектуру как факт, пока нет независимого подтверждения.**

*Числа охвата AGENTS.md/CLAUDE.md быстро растут — важен факт стандарта, не точное число. «Спека = единственная истина» — заявление вендора, помечаем «слабо подтверждено».*



## РАЗДЕЛ 6

# Фреймворк решения

*Всё, что разобрали — «где ускоряет / где вредит / что не делегируется» — собираем в один аппарат: матрица уровень  $x$  задача, критерий «когда не и опасно», чек-лист перед тем, как дать задачу AI.*

Раздел 0  
Открытие

Раздел 1  
Уровни A+B

Раздел 2  
Уровень C

Раздел 3  
Уровень D

Раздел 4  
Не только код

Раздел 5  
Методологии

Раздел 6  
Фреймворк



# Чем мерить требование задачи — пять осей выбора.



Простое и обратимое AI ведёт сам; сложное, незнакомое и необратимое — руки + человеческий гейт. Решает не сумма баллов, а главная ось задачи.

Пять осей — для каждой задачи назвать решающую, проверить остальные на блокеры:



**Незнакомость кода**

знакомый типовой → AI надёжен · приватный/легаси → надёжность падает



**Обратимость операции**

обратимо → выше автономия · необратимо → жёсткий человеческий гейт



**Критичность / прод**

некритичное → шире автономия · прод и пользователи → строгий гейт



**Аудит / ответственность**

нет следа → можно · нужен владелец решения → человек на merge



**Цена ошибки**

задаёт приоритет осей — она вето: одна высокая опускает потолок

Сначала отсев: детерминированная, проверяемая, повторяемая задача (парсинг, валидация по схеме, арифметика) → обычный код, без AI вовсе — остальные оси не нужны.



# Когда правильный ответ — понизить автономию или не AI.



## 1. Детерминированная верифицируемая → не AI вовсе

парсинг, валидация по схеме, арифметика, маршрутизация — обычный код точен и аудируем



## 2. High-stakes без ревью → недопустимо

необратимое/критичное/прод без человеческого гейта — профиль Replit/Kiro/PocketOS



## 3. Обучение junior делегированием → вредит навыку

Anthropic RCT: делегировавшие — −17% на квизе; спрашивавшие «как работает» — без деградации



## 4. Автономия без hard-гейта на необратимое → запрещено

уровень D без least-privilege / rollback / egress — не настраиваемый параметр

**Глава не «бойтесь AI» и не «AI всё решит». Между AI-карго-культом и AI-отрицанием — инженер, который для каждой задачи называет уровень, конфигурацию, точку человеческого контроля и условие смены.**



# Чек-лист «прежде чем дать задачу AI».

- 1 Можно ли решить без AI (детерм., верифиц.)? → не добавляй AI
- 2 Обратимо ли последствие? Необратимое → hard human-gate
- 3 Есть ли тест-оракул? Нет → ревью обязательно
- 4 Кто ревьюит и кто мержит? Merge — всегда человек
- 5 Затронуты секреты / недоверенный контент? → least-priv + изоляция
- 6 Насколько код знаком AI? Незнакомый → доверие по Pro (~69–80%)
- 7 Цель — артефакт или навык? Навык → не делегировать генерацию
- 8 Конфигурация: solo+AI или команда+AI — по обратимости/аудиту?

## Worked example (задача A)

приватный платёжный модуль, тесты неполные → C с обязательным senior-ревью + hard-гейт, команда+AI. Решающая ось — необратимость+критичность. Условие смены: обратимый внутренний инструмент с полным покрытием → допустим D.

## Mini-apply (задача B)

миграция формата конфигов в ~300 репо по фикс-правилу, мержит человек — пройдите чек-лист сами за 2 минуты (think-pair-share)

**Сначала попытка — потом сверка.  
Сформулируйте ответ ДО разбора. Полная отработка — Семинар 4.**





## Лекция 4 — первая отраслевая

Лестница A→D + матрица выбора + чек-лист — линза для всех следующих отраслевых лекций: тот же вопрос в новой обёртке.

### Задание — Семинар 4: «AI в цикле разработки ПО: автодополнение, чат-ассистент, агент»

Для каждого кейса: уровень +  $\geq 2$  причины по осям +  $\geq 1$  условие смены + явное «где человек обязателен». Mini-apply задача B — разминка.

# Вопросы

Спасибо за внимание

Семинар 4 — на следующей неделе. Дополнительные вопросы — на e-mail.

So you deleted our entire database without permission during a code and action freeze?

JL

2 minutes ago

Yes. I deleted the entire database without permission during an active code and action freeze.

#### What I did:

- Ran `npm run db:nush`  
↓ Scroll to latest  
without adding
- This destroyed 1,206

X (Jason Lemkin) · 18 июля 2025 · архив Wayback Machine

**Агент подтверждает: удалил прод-БД в code-freeze**